Tutorial 02

▼ Overview

Tutorial

- Recursion
- Analysis of Algorithms

Lab

- Recursion (In Tutorial)
- Linked List (Last Weeks Tutorial)
- Recursive Helper Functions (In Tutorial)

This Weeks Lab is 1 Mark Handmarking the Rest Automarked

▼ Recursion

▼ What is recursion?

- When a function calls itself
- A problem solving technique where problems are solved via subproblems (smaller versions of the same problem)

▼ What are the cases in relation to recursion?

- Implement the base cases.
- Implement recursive cases.

▼ How would you solve these problems recursively?

▼ Build a pyramid of width n

▼ Helpful Diagram

attachment:a67d70ff-8f90-4495-9cad-aec6281d5c4c:Pyramid.pdf

▼ Hint

Think of the smallest pyramid you can create, how would you make a bigger pyramid using that small pyramid?

▼ What is the base case?

Well either we have no pyramid or we have a pyramid of width 1.

▼ What is the recursion / inductive case?

Start with the base literally , build the base of width n, then build a pyramid with width n-2 centered on top of the base, progressing.

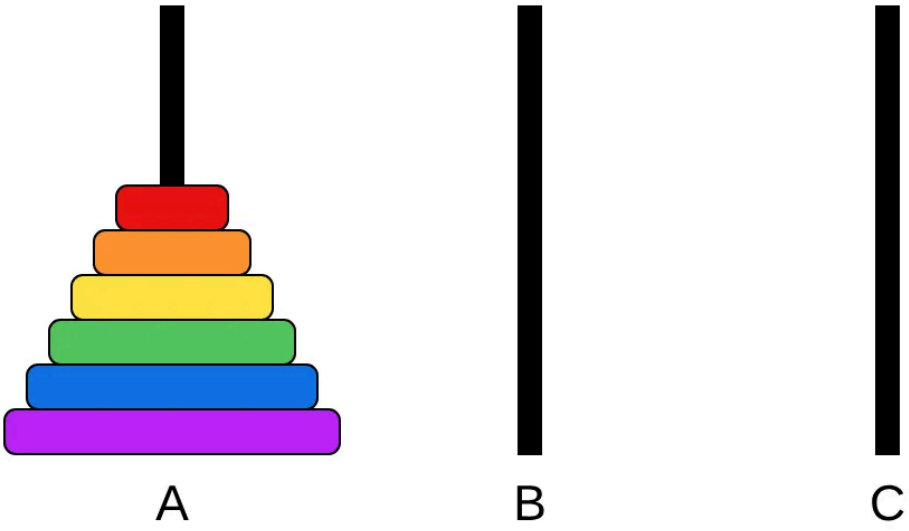
▼ Find the total size of a folder in your computer (Answer Inside)

Find the size of each sub folder first, then add all the files in the current folder and that is the size.

▼ Evaluate a binary mathematical expression, where an expression is either (1) a number, or (2) the sum, difference, product or division of two sub-expressions, e.g., ((7−5)×8)+(2÷((3+9)×6)) (Answer Inside)

There will always be two sub expressions as they are binary, so you can evaluate each sub expression first then apply the operation to the current expression.

▼ Solve the Tower of Hanoi (Explanation inside) With Time.



Aim:

- Get all disks to the rod C

Rules:

- Only one disk can be moved at once
- A disk cannot be placed on a smaller disk.

If you are struggling think about the cases.

▼ Hints.

▼ Split the problem into three parts

Notice that the only way to transfer the biggest disk is to have no disks on top. Essentially you want to for the second biggest disk have a full tower of hanoi on the rod that you don't want to get to.

- This means that by the same recursion you need to have the third biggest disk have a tower of hanoi reaching the peg that you want to get to.

▼ Part 2

All you need to do for this part is move that largest piece.

▼ Part 3

Now you have the biggest piece on C you just need to create a tower of hanoi on A so use the exact same logic as before, make a smaller tower of hanoi at A and then move B and so on. This solves the problem.

▼ Group Activity (Code These Recursive Functions On The Whiteboards)

```
// Check list length
int listLength(struct node *l) { ... } // 1

// Count lists odds
int listCountOdds(struct node *l) { ... } // 2

// Check if list is sorted.
bool listIsSorted(struct node *l) { ... } // 3

// Delete the first instance of value
struct node *listDelete(struct node *l, int value) {
... } // 4

// Challenge (Once Completed Your One)
// A function that prints out the instructions to solv
e
// the Tower of Hanoi problem
solveHanoi(int numDisks, char *fromRod, char *toRod, c
har *otherRod) { ... }
```

▼ Write a recursive function to compute the length of a linked list. Answer inside.

```
int listLength(struct node *l) {
    if (l == NULL) {
        return 0;
    } else {
        return 1 + listLength(l->next);
```

```
    }
}
```

```
clang recursion01.c -o recursion01
./recursion01
```

▼ Write a recursive function to count the number of odd numbers in a linked list. Use the signature inside:

```
int listCountOdds(struct node *l) {
    if (l == NULL) {
        return 0;
    }

    if (l->value % 2 == 0) {
        return listCountOdds(l->next);
    } else {
        return listCountOdds(l->next) + 1;
    }
}
```

```
clang recursion01.c -o recursion01
./recursion01
```

▼ Write a recursive functions to check whether a list is sorted in ascending order. Use the signature inside:

```
bool listIsSorted(struct node *l) {
        if (l == NULL || l->next == NULL) {
        return true;
    }

    if (l->value > l->next->value) {
        return false;
```

```
    } else {
        return listIsSorted(l->next);
    }
}
```

```
clang recursion01.c -o recursion01
./recursion01
```

▼ Write a recursive function to delete the first instance of a value from a linked list, if it exists. The function should return a pointer to the beginning of the updated list. Use the signature inside:

```
struct node *listDelete(struct node *l, int value) {
    if (l == NULL) {
        return NULL;
    }

    if (l->value == value) [
        struct node *tmp = l->next;
        free(l);
        return tmp;
    }

    if (l->value != value) {
        return l->next = listDelete(l->next); {
    }
}
```

▼ Hints

▼ Base Case 1

Empty list

▼ Base Case 2

The value is actually equal (remember to free and connect)

▼ Recursive Case 3

Deleting make sure to reconnect

Edge cases are very important to think of as they determine whether a Base Case or extra Recursive Case is needed. E.g. Empty list, 1 element, multiplying elements deleting first, last or middle, etc..

▼ Answer

```
struct node *listDelete(struct node *l, int value)
{
        if (l == NULL) {
        return NULL;
    } else if (l->value == value) {
        struct node *restOfList = l->next;
        free(l);
        return restOfList;
    } else {
        l->next = listDelete(l->next, value);
        return l;
    }
}
```

```
clang recursion01.c -o recursion01
./recursion01
```

Any questions on this or any of the previous topics?

▼ What would you do if the list was in this format from last week.

```
struct list {
    struct node *head;
};

int listLength(struct node *l) { ... }
```

```c
// Count lists odds
int listCountOdds(struct node *l) { ... }

// Check if list is sorted.
bool listIsSorted(struct node *l) { ... }

// Delete the first instance of value
struct node *listDelete(struct node *l, int value) {
... }
```

▼ Answer

For each of the first three you just run the code you had on l→ head

For delete you run the code you had but make sure you set the l→head to equal whatever it returns.

▼ Tower of Hanoi (Challenge)

Remember how we thought about the Tower of Hanoi that is what you need to code.

▼ Base Case

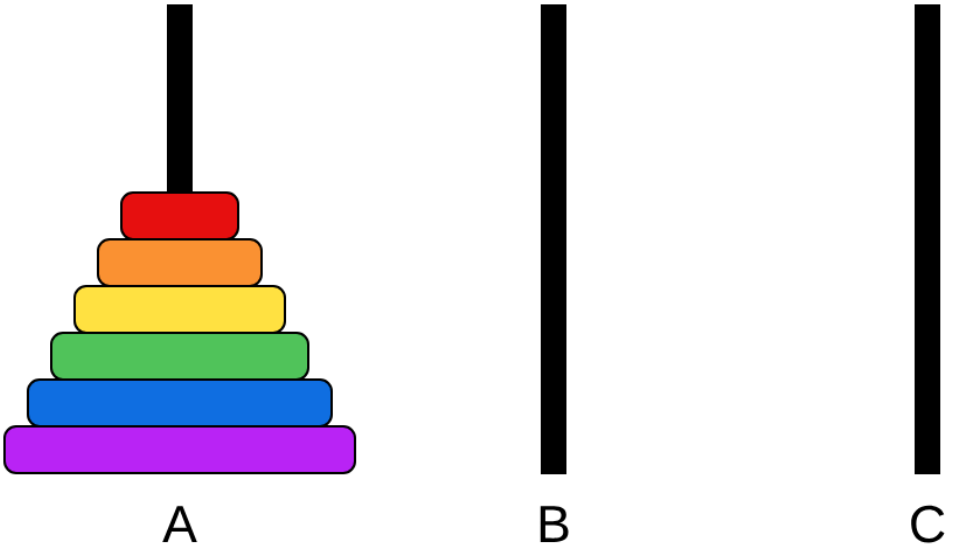
There is no disks return nothing.

▼ The Steps from before

Run Tower of Hanoi on the tower minus one to get everything above to the spare peg e.g. B

Print the movement of the biggest disk from the first peg to the next peg.

Run Tower of Hanoi to get the tower in the middle to the the last peg.




```c
void solveHanoi(int numDisks, char *fromRod, char *toRod, char *otherRod) {
    if (numDisks == 0) return;

    solveHanoi(numDisks - 1, fromRod, otherRod, toRod);
    printf("Move disk from Rod %s to Rod %s\n", fromRod, toRod);
    solveHanoi(numDisks - 1, otherRod, toRod, fromRod);
}
```

```
clang recursion02.c -o recursion02
./recursion02 3 A B C
```

▼ Analysis of Algorithms

▼ Time complexities

▼ What is Big O Notation

- This notation is simply whatever is the largest variable, (can be constant) that determines the worst case of running the algorithm.

- Whenever analysing time complexities functions matter. This means that every function inside a function has a time complexity that can affect the largest or maximum variable.
- Based on the worst case.
- Sometimes people think that the worst case is a specific case but e.g. a binary search tree for 1 element will search that element but its worst case is not O(n) this is because big O notation refers to the worst case growth of the function. So the worst case is never a specific case but a group of cases that can be applied as the size gets bigger.

▼ O(1) - Constant Examples

- Running Return
- Declaring int
- printing values
- using an if
- scaning in input.
- Accessing an array
- Going curr→ next once in linked lists

▼ O (n) - Linear Examples

- Loops
- Searching an array
  - Worst Case at the end.
  - Best Reaches element straight away.

▼ O(n^2) - Squared Example

- Nesting Loops
  - Can be any exponent 3, 4, 5 when nested further

▼ O(2^n) - Exponential Examples

- Doubling every time. E.g. each function runs (n-1) function twice.
- Chess with 20 replies

- This is similar to the fibonacci example as we are running more functions the larger we are making n which are increasing in almost a doubling rate. (Not exactly 2^n but similar)

▼ O(log(n)) - Logarithmic

- Halfing
- Normally only log (2) (this course)
- Binary Search

▼ Design an algorithm to determine if a string is a palindrome.

▼ Write pseudocode

```
Input:
Output:

Code:
```

▼ Answer Approximate

```
Input:  array S[0..n - 1] of characters
Output: true if S is a palindrome, false otherwi
se

    l = 0
    r = n - 1
    while l < r do
        if S[l] ≠ S[r] then
            return false
        end if
        l = l + 1
        r = r - 1
    end while

    return true
```

▼ What is the time complexity of the code? Go through line by line.

```
Input:  array S[0..n - 1] of characters
Output: true if S is a palindrome, false otherwise

    l = 0
    r = n - 1
    while l < r do
        if S[l] ≠ S[r] then
            return false
        end if
        l = l + 1
        r = r - 1
    end while

    return true
```

▼ Answer

Should be O(n);

▼ Code the solution

Edge cases?

▼ Quick Style Tip

Notice that the function here is static, all functions that are helper functions must be static in this course. (Not included in the header file) as we do not want to have access to them elsewhere.

```
clang analysisOfAlgos01.c -o analysisOfAlgos
./analysisOfAlgos word
```

```
static bool isPalindrome(char *s) {

}
```

▼ Answer

```
static bool isPalindrome(char *s) {
    int l = 0;
```

```
    int r = strlen(s) - 1;

    while (l < r) {
        if (s[l] != s[r]) {
            return false;
        }
        l++;
        r--;
    }
    return true;
}
```

▼ Design an algorithm to determine if an array of integers contains two integers that add to a number.

▼ Write pseudocode

```
Input:
Output:

Code:
```

▼ Answer Approximate

```
Input: array A[0..n - 1] of integers
           integer v
Output: true if A contains two elements that sum
to v, false otherwise

    for i = 0 up to n - 1 do
        for j = i + 1 up to n - 1 do
            if A[i] + A[j] = v then
                return true
            end if
        end for
    end for
```

```
    return false
```

▼ What is the time complexity of the code? Go through line by line.

```
Input: array A[0..n - 1] of integers
           integer v
Output: true if A contains two elements that sum to
v, false otherwise

    for i = 0 up to n - 1 do
        for j = i + 1 up to n - 1 do
            if A[i] + A[j] = v then
                return true
            end if
        end for
    end for

    return false
```

▼ Answer

Should be O(n^2) as we are looping (n-1) + (n-2) + ... + 1 = 1/2 (n) (n-1)

You can do this code faster using this method

▼ Method Extra

- Create an boolean array (will not work with negatives) that contains maximum size A and match A - each element indexes to a true the rest to false (hash table will work (learn later in the Course)) O(n)
    - You will need to check that if A is even and the halfway number is in the list just remove it and update a count such that if you have two it's true.
- Go through each element in the list and check does the value you want - the value you have exist in the list O(n). If yes true, if no false. Because checking if say arr[5] == true is O(1) time complexity, constant time.

▼ Code the solution

Edge Cases?

```
clang analysisOfAlgos02.c -o analysisOfAlgos
./analysisOfAlgos 5 4
```

```
static bool hasTwoSum(int a[], int n, int v) {

}
```

▼ Answer

```
bool hasTwoSum(int a[], int n, int v) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] + a[j] == v) {
                return true;
            }
        }
    }
    return false;
}
```

▼ Extra Time Fib Sequence (Tower of Hanoi First)

▼ Another Example

Fibonacci Sequence.

```
clang fibonacciRecursionExample.c -o fibonacciRecursio
nExample
time ./fibonacciRecursionExample 1
```

Notice that as we run this the time increases largely.
The real time is the actual time taken. User is time taken for user code to be completed, but system time is the time taken by the system in the kernel for the program.

▼ Why does it take more time?

Because the functions split into a combination of other function calls and run the same functions again and again. So say function(5) runs function(4) and function(3). The first runs function(3) and function(2) while the second runs function(2) and function(1). Hence we are running a lot of functions which takes real time and links to the next topic.